

程式碼實驗室：運用 Gemini 加速以測試為導向的開發作業

來源：<https://codelabs.developers.google.com/tdd-ruby-app-duetai?hl=zh-tw>

步驟數：10

我們會請 Gemini 協助使用 TDD (測試導向開發) 方法建立簡單的 Ruby 應用程式。

目錄

1. [1. 簡介](#)
2. [2. 必要條件](#)
3. [3. Gemini 熱身](#)
4. [4. 專案設定](#)
5. [5. Gemini 驅動的實作](#)
6. [6. 使用 Gemini 進行疊代式 TDD](#)
7. [7. 將應用程式部署至 Cloud Functions](#)
8. [8. Ruby 適用的 Functions Framework](#)
9. [9. 端對端測試](#)
10. [10. 結論](#)

步驟 1 · 約 0 分鐘

1. 簡介

本程式碼研究室將介紹測試驅動開發 (TDD) 的原則，也就是先編寫測試，再實作程式碼。我們會使用 Ruby 建構簡單的計算機應用程式，展示這種做法。

您也會瞭解如何使用 Gemini 做為程式碼助理。Gemini 可生成測試案例、建議程式碼實作方式，甚至提供程式碼說明，有助於加快 TDD 週期。

最後，您將瞭解如何將計算機應用程式部署至 Google 的全代管無伺服器平台 Cloud Functions，讓使用者可以存取應用程式。

2. 必要條件

- 已啟用計費功能的 Google Cloud 專案
- 網路瀏覽器

建立專案

- 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
- 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟用 Gemini API

- 前往 [Gemini Marketplace](#) 啟用 API。您也可以使用 gcloud 啟用 API：

```
gcloud services enable cloudaicompanion.googleapis.com --project PROJECT_ID
```

- 前往 [Gemini 頁面](#)，然後按一下「開始對話」。
- 重要事項：請按照這個[程式碼實驗室](#)的步驟 1 和 2 操作，分別開始使用 Gemini，並在 [Cloud Shell IDE](#) 中啟用 Gemini。

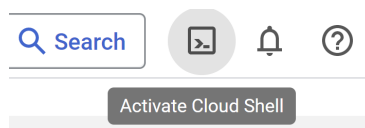
啟用 Cloud Functions API

- 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
- 前往 [Cloud Functions API 頁面](#) 啟用 API。您也可以使用 gcloud 啟用 API：

```
gcloud services enable cloudfunctions.googleapis.com --project PROJECT_ID
```

啟用 Cloud Shell

- 您將使用 [Cloud Shell](#)，這是 Google Cloud 控制台執行的指令列環境，已預先載入必要語言。在 Cloud 控制台，點選右上角的「啟用 Cloud Shell」：



- 連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

- 在 Cloud Shell 中執行下列指令，確認 gcloud 指令知道您的專案：

```
gcloud config list project
```

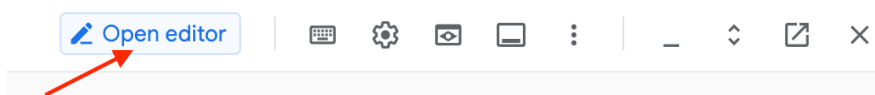
- 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

開啟編輯器

- 在本程式碼研究室中，我們將使用內建的 Cloud 編輯器。在 Cloud Shell 視窗中，按一下右上角的「開啟編輯器」按鈕。系統會開啟 VSCode 編輯器。

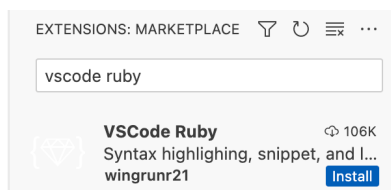


新增 Ruby 擴充功能

- 在左側選單中，按一下擴充功能圖示



，在搜尋列中輸入 `vscode ruby`，然後安裝 VSCode Ruby 擴充功能。



3. Gemini 熱身

有時 Gemini 對話的回覆可能過於具體，或您想針對特定回覆重新開始。這時請使用「重設對話」/「清除對話」功能。

LLM 屬於非確定性模型，因此，您嘗試這些提示詞時，收到的回覆可能與程式碼研究室隨附螢幕截圖中的回覆不同。

首先，請問問 Gemini 什麼是 TDD。如要在同一個 Cloud Shell 編輯器中與 Gemini 對話，請按照下列步驟操作：

- 在頁面左側找到 Gemini 圖示



，即可開啟左側的對話視窗。

- 在顯示「Enter a prompt here」的文字方塊中輸入內容

Enter a prompt here



提示：

What is Test Driven Development?

回應：

Test-driven development (TDD) is a software development methodology that requires developers to write automated tests before writing any production code. This helps ensure the code is correct and meets the requirement.

....

簡要介紹 TDD 及其優點。

現在來問問 Gemini Ruby 是什麼：

提示：

What is Ruby?

回應：

Ruby is a general-purpose, interpreted, object-oriented programming language. It is designed to be easy to read and write. Ruby is dynamically typed, meaning that the type of a variable is not declared and can change during execution.

....

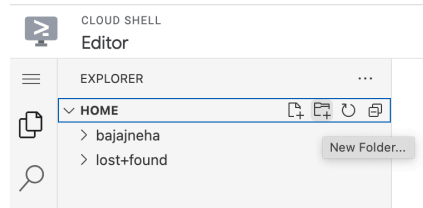
本文將說明 Ruby 的用途，以及使用 Ruby 做為語言的好處。

步驟 4 · 約 0 分鐘

4. 專案設定

現在開始建立簡單的 Sinatra 應用程式。

- 現在建立一個空資料夾。命名方式不限。在本程式碼研究室中，我們將其命名為 `gemini-tdd`。在左側的雲端編輯器中，於「Explorer」分頁下方，在 `/home/<YOUR-USERNAME>/` 下方建立資料夾。

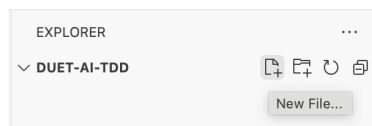


- 開啟剛建立的資料夾。在最左側的選單中，按一下圖示



，然後依序按一下「File」->「Open Folder」。

- 我們要在這個根資料夾中建立新的 Ruby 檔案，做為應用程式的起點，您可以隨意命名。在本程式碼研究室中，我們將其命名為 `app.rb`。



- 我們也要在根資料夾中新增 `Gemfile`。建議您也問問 Gemini `Gemfile`。
 - 我們也會建立 `app_test.rb` 檔案來編寫測試案例。
-

5. Gemini 驅動的實作

現在，請先向 Gemini 提供我們想執行的作業背景資訊，然後要求 Gemini 為該作業生成測試案例。在 Gemini 對話提示中複製下列提示。

提示：

```
Answer as a software engineer with Ruby expertise. We want to build a Ruby calculator application with a test driven development approach. Let's start by writing some tests for the calculator app using minitest gem.
```

回應：

```
Here is a simple test for a calculator app that adds two numbers:
```

```
require "minitest/autorun"

describe "Calculator" do
  it "should add two numbers" do
    calculator = Calculator.new
    assert_equal 5, calculator.add(2, 3)
  end
end
```

```
This test uses the assert_equal method to verify that the result of the add method is correct.
.....
```

其中提供計算機應用程式的範例測試案例，也就是新增兩個數字。此外，也匯入了測試所需的依附元件 `minitest/autorun`。請將內容複製到 `app_test.rb`。現在來試著執行這項測試；由於我們沒有編寫任何程式碼，測試一定會失敗。

我們來問問 Gemini 如何執行這項測試。

提示：

```
How do we run the tests written in app_test.rb using ruby command?
```

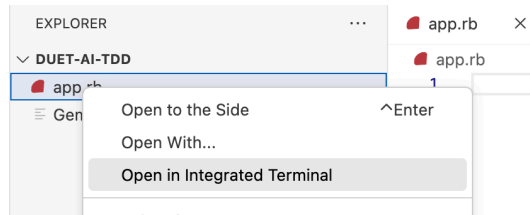
回應：

```
To run the tests written in app_test.rb using the ruby command, you can use the following steps:
1.Make sure that you are in the same directory as the app_test.rb file.
2.Open a terminal window.
3.Type the following command:
```

```
ruby app_test.rb
```

This will run all of the tests in the `app_test.rb` file and report the results.

接著執行測試。在「Explorer」分頁標籤中，對 `app_test.rb` 按一下滑鼠右鍵，然後點選「Open in Integrated Terminal」(在整合式終端機中開啟)



確認您位於與檔案 `app_test.rb` 相同的資料夾中，如果不是，請使用 `cd` 指令前往該資料夾。在終端機執行 Gemini 傳回的指令：

```
ruby app_test.rb
```

您會遇到下列錯誤：

```
NameError: uninitialized constant Calculator
```

我們直接在編輯器中詢問 Gemini 如何修正。按一下 Cloud Shell 編輯器的導覽窗格中的對話圖示，然後輸入下列提示：

提示：

```
NameError: uninitialized constant Calculator. Give me Calculator class code.
```

回應：

```
The error you are getting is because you have not defined the Calculator class. You
need to define the Calculator class before you can use it in your tests.
Here is the code for the Calculator class:
```

```
Class Calculator
  def add(a, b)
    a + b
  end
end
```

```
Once you have defined the Calculator class, you can run the tests again and they will
pass.
```

```
.....
```

Gemini 告訴我們沒有名為 `Calculator` 的類別，因此我們需要建立一個。此外，系統也提供建立 `Calculator` 類別的程式碼。我們就使用這段程式碼吧。從「Explorer」分頁開啟 `app.rb`，並複製 Gemini 提供的程式碼。在 `app.rb` 中修改內容時，Gemini 可能會提供自動完成建議；如要接受建議，請按 `tab`，如要拒絕，請按 `Esc`。

現在，請再次執行測試。在整合式終端機中輸入下列指令：

```
ruby app_test.rb
```

但您仍會遇到相同的「`NameError: uninitialized constant Calculator`」錯誤。我們來問問 Gemini 如何修正。

步驟 6 · 約 0 分鐘

6. 使用 Gemini 進行疊代式 TDD

提示：

```
I am still getting this error: "NameError: uninitialized constant Calculator".
```

回應：

The error you are getting is because you have not defined the Calculator class. You need to define the Calculator class before you can use it in your tests. Here is the code for the Calculator class:

```
class Calculator
  def add(a, b)
    a + b
  end

  def subtract(a, b)
    a - b
  end

  def multiply(a, b)
    a * b
  end

  def divide(a, b)
    a / b
  end
end
```

....

這會提供一些回應，可能與上述回應相同，也可能不同。因此，我們明確告知 Gemini，Calculator 類別是在 app.rb 中定義。

提示：

```
Calculator class is defined in app.rb already.
```

回應：

I see that the Calculator class is defined in the app.rb file. However, you are getting the NameError error because you are trying to use the Calculator class in the app_test.rb file without requiring the app.rb file. To fix this, you need to add the following line to the top of the app_test.rb file:

```
require_relative "app"
```

我們收到 Gemini 的回覆，可以匯入定義了 Calculator 類別的 app.rb。有時可能需要 2 到 3 次的提示迭代，才能獲得正確的回覆，但只要掌握訣竅，就能越來越得心應手。請注意，Gemini 的回覆也可能顯示 ***require "./app"***。請將這項內容新增至 app_test.rb 中的 app.rb，然後再次執行相同測試。

請確認此時的 app_test.rb 如下所示：

```
require "minitest/autorun"
require "./app"

describe "Calculator" do
  it "should add two numbers" do
    calculator = Calculator.new
    assert_equal 5, calculator.add(2, 3)
  end
end
```

在專案資料夾中，透過 Cloud Shell 終端機執行下列指令：

```
ruby app_test.rb
```

如果一切順利，測試應該會通過。因此，我們使用 TDD 方法開發了非常簡單的 Ruby 應用程式，並請 Gemini 協助。提示回覆每次都不會相同，這是 LLM 模型的天性。Gemini 可能會提供更多計算機操作，但整體而言，我們剛才執行的程序應該適用。

步驟 7 · 約 0 分鐘

7. 將應用程式部署至 Cloud Functions

接著問問 Gemini 將這個函式部署至 Cloud Functions：

提示：

```
Let's deploy this app to Cloud Functions
```

回應：

```
To deploy your app to Cloud Functions, you can use the following steps:
1.Create a new Cloud Functions project.
2.Create a new Cloud Function.
3.Set the trigger for the Cloud Function to be an HTTP request.
4.Set the entry point for the Cloud Function to be the app method in the app.rb file.
5.Deploy the Cloud Function.
Once you have deployed the Cloud Function, you can test it by sending an HTTP request
to the function's URL.
....
```

Gemini 會說明如何使用 Cloud Functions UI 手動部署。但我不想這麼做。請 Gemini 提供 `gcloud` 指令，我可以直接從整合式終端機執行。

提示：

```
How do I deploy this app to Cloud Functions gen2 using the gcloud command?
```

回應：

```
...
Here are the specific commands you can use to deploy your app to Cloud Functions
using the gcloud command:

gcloud functions deploy calculator --runtime ruby32 --trigger-http --entry-point app

This command will create a new Cloud Function called calculator that is triggered by
an HTTP request and uses the app method in the app.rb file as the entry point.

Once the Cloud Function is deployed, you can test it by sending an HTTP request to
the function's URL. For example, you can use the following command to send a request
to the function:

curl -X POST -H "Content-Type: application/json" -d '{"a": 2, "b": 3}'
https://REGION-PROJECT_ID.cloudfunctions.net/calculator
```

在終端機中執行 Gemini 提供的 `gcloud` 指令。

```
gcloud functions deploy calculator --runtime ruby32 --trigger-http --entry-point app
```

部署應用程式時，如果系統提示允許未經驗證的要求，請輸入「yes」，因為本程式碼研究室不會處理驗證作業。

您會遇到下列錯誤：

```
「ERROR: (gcloud.functions.deploy) OperationError: code=3, message=Build failed: Could not find
Gemfile.lock file in your app. Please make sure your bundle is up to date before deploying.;
Error ID: 7916f05c」
```

讓我們請 Gemini 幫忙修正錯誤。

提示：

```
ERROR: (gcloud.functions.deploy) OperationError: code=3, message=Build failed: Could
not find Gemfile.lock file in your app. Please make sure your bundle is up to date
before deploying.; Error ID: 7916f05c
```

回應：

```
It looks like you're trying to deploy a function to Cloud Functions, but you don't
have a Gemfile.lock file in your app. To fix this, run the following command in your
app directory:
```

```
bundle install
```

```
This will create a Gemfile.lock file that lists all of the gems that your app uses.
...
```

讓我們在終端機中執行 `bundle install`，這會建立 `Gemfile.lock`。執行指令後，請檢查「Explorer」分頁，您應該會看到 `Gemfile.lock`。

請再次嘗試部署至 Cloud Functions。在終端機中執行相同的部署指令：

```
gcloud functions deploy calculator --runtime ruby32 --trigger-http --entry-point app
```

您會遇到下列其他錯誤：

```
「ERROR: (gcloud.functions.deploy) OperationError: code=3, message=Build failed: unable to
execute functions-framework-ruby; please ensure a recent version of the functions_framework gem
is in your Gemfile; Error ID: 89ae7dda」
```

步驟 8 · 約 0 分鐘

8. Ruby 適用的 Functions Framework

我們來問問 Gemini 關於這項錯誤：

提示：

```
ERROR: (gcloud.functions.deploy) OperationError: code=3, message=Build failed: unable to execute functions-framework-ruby; please ensure a recent version of the functions_framework gem is in your Gemfile; Error ID: 89ae7dda
```

回應：

It looks like you're trying to deploy a function to Cloud Functions, but the build is failing because the `functions_framework` gem is not in your Gemfile. To fix this, add the following line to your Gemfile:

```
gem "functions_framework"
```

Then, run the following command to install the gem:

```
bundle install
```

Once the gem is installed, you can try deploying your function again.

....

因此，應用程式中缺少 `functions_framework` gem。請在 Gemfile 中新增下列程式碼：

```
gem "functions_framework"
```

現在請在終端機中執行 `bundle install`。您可能會遇到「No explicit global resource in Gemfile」的錯誤訊息，也可能不會。如果遇到這個問題，請確認 Gemfile 的第一行是下列程式碼：

```
source "https://rubygems.org"
```

然後再次從終端機執行 `bundle install`。現在，請再次執行部署指令：

```
gcloud functions deploy calculator --runtime ruby32 --trigger-http --entry-point app --project PROJECT_ID
```

現在會遇到下列錯誤：

```
「ERROR: (gcloud.functions.deploy) OperationError: code=3, message=Build failed: failed to verify function target "app" in source "app.rb": Undefined function: "app"; Error ID: e12ab0f9」
```

為解決這個問題，請嘗試使用 Gemini 的 `Generate Code` 功能。從「Explorer」分頁開啟 `app.rb`，其中已有計算機應用程式的程式碼。在 `app.rb` 的第一行中，開始輸入 `FunctionsFramework`，然後輸入「`.`」。您會看到以灰色文字顯示的部分建議：

```
FunctionsFramework.http "add" do |request|
```

按下 Tab 鍵即可接受建議。接受後，你會看到更多建議，如下所示：

```
FunctionsFramework.http "add" do |request|  
  a = request.params["a"].to_i  
  b = request.params["b"].to_i  
  Calculator.new.add(a, b)  
end
```

按下 Tab 鍵即可接受其他建議。因此，您的 app.rb 應如下所示：

(將函式名稱從 *add* 重新命名為 *app*)

```
require "functions_framework"  
  
FunctionsFramework.http "app" do |request|  
  a = request.params["a"].to_i  
  b = request.params["b"].to_i  
  Calculator.new.add(a, b)  
end  
  
class Calculator  
  def add(a, b)  
    a + b  
  end  
end
```

我們剛才建立了 Cloud Functions 應用程式的進入點，也瞭解如何在編輯器中使用 Gemini 的程式碼建議。現在，請嘗試再次部署應用程式。

```
gcloud functions deploy calculator --runtime ruby32 --trigger-http --entry-point app --  
project PROJECT_ID
```

如果一切順利，部署作業應該會完成，您也應該能看到應用程式的代管網址。很棒，現在稍微修改 app.rb，傳回加法結果。最終的 app.rb 應如下所示：

```
require "functions_framework"

FunctionsFramework.http "app" do |request|
  a = request.params["a"].to_i
  b = request.params["b"].to_i
  calculator = Calculator.new
  add = calculator.add(a, b)
  "Result: #{add}"
end

class Calculator
  def add(a, b)
    a + b
  end
end
```

步驟 9 · 約 0 分鐘

9. 端對端測試

開啟網址並傳遞查詢參數 `a` 和 `b`，如下所示：

```
https://[APP-URL]/?a=2&b=3
```

您應該會看到類似 `"Result: 5"` 的回應。計算機可以執行許多運算，您可以擴充這個應用程式，讓它也能執行這些作業，並在過程中尋求 Gemini 的協助。

10. 結論

我們今天瞭解如何使用 TDD 方法，在 Gemini 的協助下開發 Ruby 應用程式。如您所見，Gemini 可能會提供不同的結果，但絕對能協助您加快開發程序，即使您不熟悉某種語言，Gemini 仍可顯示函式或程式碼片段，加快您的學習速度。

無論經驗如何，開發人員都能在開發過程中尋求 Gemini 的協助。如果不是為了開發，你可以使用 Gemini 解釋現有程式碼，也可以要求 Gemini 為現有程式碼生成測試，這至少是我最無聊的工作 😊

祝您使用 Gemini 順利完成程式碼！
